

Co wypisze ten kod? Co zmieni zamiana **var** na **let**?

```
for (var i = 0; i < 3; i += 1) {  
  const label = `krok ${i}`;  
}  
  
console.log(i);
```

Która linia rzuci błąd i dlaczego? Czego pilnuje **const**?

```
const settings = { theme: "dark" };  
  
settings.theme = "light";  
settings = { theme: "light" };
```

Co wypisze każde **typeof**? Jak rozpoznać tablicę i brak wartości?

```
console.log(typeof ["Laptop"]);  
console.log(typeof null);  
console.log(typeof (() => {}));
```

Co wypisze ostatnia linia i dlaczego? Jak to naprawić?

```
const customerName = "anna";  
  
customerName.toUpperCase();  
console.log(customerName);
```

Co wypisze każda linia? Czym to grozi przy pustym polu formularza?

```
console.log(Number(""), Number(null), Number(undefined));  
console.log("5" + 1, "5" - 1);
```

Ile elementów zostanie i które odpadną? Czym są **falsey values**?

```
const values = [0, "0", "", [], null, NaN, "false"];  
  
console.log(values.filter(Boolean).length);
```

Co wypisze każde porównanie?
Dlaczego w kursie używamy tylko ===?

```
console.log("" == 0, 5 == "5");  
console.log(null == undefined, null === undefined);
```

Co wypisze każda linia? Jak poprawnie wykryć **NaN**?

```
const pricePln = Number("12,50");  
  
console.log(pricePln === NaN, typeof pricePln);  
console.log(Number.isNaN(pricePln));
```


Co wypisze każda linia? Który wynik jest poprawny dla klienta bez rabatu?

```
const discountPercent = 0;  
  
console.log(discountPercent || 10);  
console.log(discountPercent ?? 10);
```

Co wypisze każda linia? Co dokładnie zwracają `||` i `&&`?

```
console.log(0 || "" || 42 || "Laptop");  
console.log(20 && "Laptop" && 0 && 42);
```

Co wypisze każda linia? Przed czym ?. NIE chroni?

```
const order = { id: "ORD-1002", customer: null };  
  
console.log(order.customer?.address?.city);  
console.log(order.discount?.percent * 2);
```

Co wypisze ten switch dla **"new"**? A
co dla wartości **"2"** przy **case 2**?

```
switch (orderStatus) {  
  case "new": console.log("nowe");  
  case "paid": console.log("opłacone"); break;  
  default: console.log("inne");  
}
```

Dlaczego ten kod się nie uruchomi? Czym zastąpić **forEach**?

```
orderId.forEach((orderId) => {  
    if (orderId === "ORD-3") break;  
    console.log(orderId);  
});
```

Co wypisze każde wywołanie? Kiedy naprawdę wchodzi parametr domyślny?

```
const greet = (name = "Gościu") => `Witaj, ${name}`;  
console.log(greet(undefined), greet(null), greet(""));
```

Co wypisze każde wywołanie?
Gdzie tu jest **rest**, a gdzie **spread**?

```
const sum = (...values) => values.reduce((a, b) => a + b, 0);  
const pricesPln = [10, 20, 30];  
  
console.log(sum(pricesPln), sum(...pricesPln));
```

Jaki motyw wyjdzie i dlaczego? Jak to poprawić?

```
const defaults = { theme: "dark", lang: "pl" };  
const userPrefs = { theme: "light" };  
  
console.log({ ...userPrefs, ...defaults });
```


Co jest w **safeUser**? Gdzie taki zapis przydaje się komercyjnie?

```
const user = { id: 7, email: "a@b.pl", passwordHash: "x1" };  
  
const { passwordHash, ...safeUser } = user;
```

Co wypisze każda zmienna? Jakie dwa błędy siedzą w linii z **sort**?

```
const pricesPln = [3, 25, 100];  
  
const sorted = pricesPln.sort();  
console.log(sorted, pricesPln);
```

Co wypisze każda linia? Kiedy sięgamy po **map**, a kiedy po **reduce**?

```
const cart = [{ pricePln: 100, qty: 2 }, { pricePln: 50, qty: 1 }];  
  
console.log(cart.map((item) => item.pricePln * item.qty));  
console.log(cart.reduce((sum, item) => sum + item.pricePln, 0));
```

Co wypisze każda linia? Podaj komercyjne zastosowanie **Set** i **Map**.

```
const visitedPaths = ["/home", "/cart", "/home"];  
console.log([...new Set(visitedPaths)]);  
  
const pageHits = new Map([["/home", 2]]);  
console.log(pageHits.get("/cart"), pageHits.size);
```